

Reversing tutorial

Jeff Epstein

1 Introduction

According to Wikipedia, *reversing*, also known as *reverse engineering*, is the processes of extracting knowledge or design information from anything man-made.

Specifically, we are interested in understanding how a program works, and possibly in modifying it. If you have the program's source code available, you can just read it, but if you don't, you have to resort to more drastic measures.

2 Procedure

2.1 Setup

If you're using Anubis or Vital, the necessary tools should already be available. Otherwise, you can install them by running this command in your Linux machine:

```
sudo apt-get install gdb hexedit nasm binutils
```

If you're not using Anubis, then you'll need to download the provided `gdbinit` file from Brightspace and save it in your home directory (typically `/home/user` or similar) as `“.gdbinit”`. Note that the file name must begin with a period, i.e. the name is period, then the seven letters `gdbinit`. Anubis should already have this file installed.

In this tutorial, we'll be working with a simple “Hello, World” application as an example. Download the assembly program source code, `helloworld.asm`. Assembly language source code typically has a suffix `.asm` or `.s`. Read the file and make sure you understand it.

2.2 Creating a program

To *assemble* a program means to (mechanically) translate it from assembly language into the corresponding machine code, which can be executed by the processor. The *assembler* (analogous to a compiler) is a program that performs this operation. We'll be using the assembler `nasm`.

Assemble and link the source code with the following commands:

```
nasm -f elf helloworld.asm
ld -s -o helloworld -m elf_i386 helloworld.o
```

If these commands fail, they'll display an error message. If they succeed, they will display nothing.

The above commands will produce a binary executable file (also referred to as a *binary* or an *executable*) named `helloworld`. At this point, we don't need the assembly source any more, and you can delete it: future operations depend only on the executable file.

Now, let's run the program to make sure it does what we think it does.

```
./helloworld
```

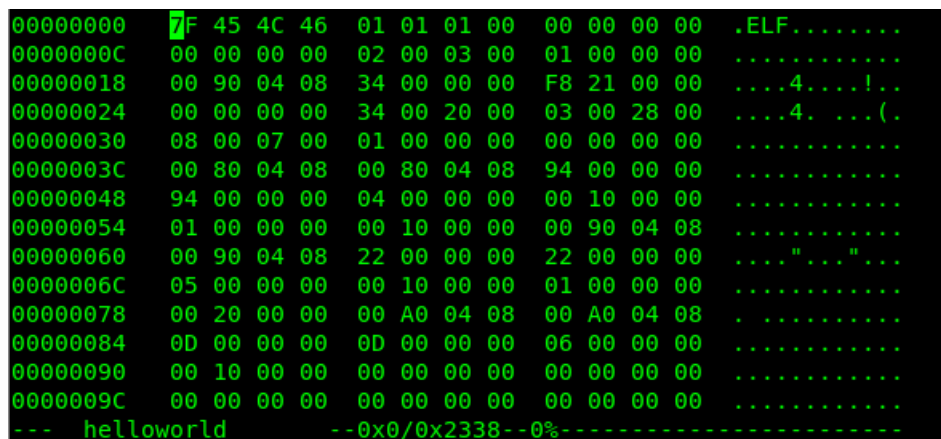
You should see the text “Hello, world”.

We can view the raw contents (i.e. a *dump*) of the executable using the command:

```
hexedit helloworld
```

(You can also use `ghex` instead of `hexedit`, if you have it.)

It might look like this:



```
00000000  7F 45 4C 46 01 01 01 00 00 00 00 00 .ELF.....
0000000C  00 00 00 00 02 00 03 00 01 00 00 00 .....
00000018  00 90 04 08 34 00 00 00 F8 21 00 00 ....4....!..
00000024  00 00 00 00 34 00 20 00 03 00 28 00 ....4. ...(.
00000030  08 00 07 00 01 00 00 00 00 00 00 00 .....
0000003C  00 80 04 08 00 80 04 08 94 00 00 00 .....
00000048  94 00 00 00 04 00 00 00 00 10 00 00 .....
00000054  01 00 00 00 00 10 00 00 00 90 04 08 .....
00000060  00 90 04 08 22 00 00 00 22 00 00 00 ...."...."....
0000006C  05 00 00 00 00 10 00 00 01 00 00 00 .....
00000078  00 20 00 00 00 A0 04 08 00 A0 04 08 . ....
00000084  0D 00 00 00 0D 00 00 00 06 00 00 00 .....
00000090  00 10 00 00 00 00 00 00 00 00 00 00 .....
0000009C  00 00 00 00 00 00 00 00 00 00 00 00 .....
--- helloworld --0x0/0x2338--0%-----
```

The file is displayed in three columns: the rightmost column is the ASCII representation of the content of the file. The middle column is the same data, now expressed as a sequence of hex bytes. Finally, the leftmost column specifies the offset (i.e. the position in the file) of that row.

Scroll around (with the arrow keys) and explore the file. Notice that in the dump you can see some familiar landmarks:

- The file begins with the “ELF” signature, indicating that this is an Executable and Linkable Format file.
- The file contains the literal string “Hello, world”. Can you find it?
- The file contains marking for the `.text` (i.e. code) and `.data` sections of the file (explained below). Can you find them?

Other than the above landmarks, the raw content of the executable file should be opaque to you, displayed as a bunch of meaningless symbols. Nevertheless, we know that the file content describes the behavior of the program, just as our source code did: the executable is in fact a translation of the source. Actually, the meaningless symbols are the ASCII representation of machine code.

Also note that the whole file is very small. On my system, it’s just 360 bytes. Consider that most programs these days are several megabytes at least. Assembly language has the capability to create very efficient programs, both in terms of speed and memory.

2.3 Looking closer at the executable

Although it’s not visible to the naked eye from the raw dump, all ELF executable files are divided into an arbitrary number of named *sections*, which can be examined individually. Although a program may consist of many sections, there are only a few that are interesting to us:

- `.text` – The actual executable portion of the program, consisting of machine code instructions in binary. When the program is loaded (i.e. when you tell the OS to run the program), the content of this section is copied into memory. The OS then basically does a `jmp` to the address of the first instruction of the program, thus starting execution.

Don’t be confused: `.text` is, for historical reasons, the name for the code section of the executable, but don’t expect to find readable text there.

- **.data** – Non-executable, initialized data (that is, data that already has a value when the program starts). This is typically used for strings, arrays, and other large data types that can’t be encoded directly into an assembly instruction. When the program is loaded, the content of this section is copied into memory, as well. These values are then accessed by the code at their address.
- **.bss** – Similar to **.data**, but this section represents data that is uninitialized, meaning that it has no value when the program starts: the code must assign it a value. Since values in this section have no initial value, there’s nothing to put in the executable file. Instead, it’s sufficient if the executable knows the size (in bytes) of the section, so that the OS can allocate sufficient uninitialized memory when the program is loaded. Like the **.data** section, data here is accessed by its address.

The name of this section is also historical: originally, BSS stood for “Block Started by Symbol.”

In this case, our program has only **.data** and **.text** sections.

2.4 Disassembly

To *disassemble* a program is to convert its executable, binary representation into assembly language, i.e. the reverse of assembling it. This is desirable so that we can figure out what a program is doing, even if we don’t have its source code. We can get a disassembly of the executable sections of the **helloworld** program with this command:

```
objdump -d -Intel helloworld
```

I get the following listing:

```
helloworld:      file format elf32-i386
```

Disassembly of section **.text**:

```
08048080 <.text>:
8048080: b8 04 00 00 00      mov     eax,0x4
8048085: bb 01 00 00 00      mov     ebx,0x1
804808a: b9 a4 90 04 08      mov     ecx,0x80490a4
804808f: ba 0d 00 00 00      mov     edx,0xd
8048094: cd 80              int     0x80
8048096: b8 01 00 00 00      mov     eax,0x1
804809b: bb 00 00 00 00      mov     ebx,0x0
80480a0: cd 80              int     0x80
```

Let’s look at some important features here:

- At the top, it tells us that it’s a 32-bit ELF executable built to run on i386-class processors (meaning, 32-bit Intel architecture processors, as opposed to 16-bit or 64-bit Intel architecture processors or processors of other architectures, such as MIPS, ARM, or PowerPC). Trying to run this program on incompatible hardware wouldn’t work, since each family of processors has a different instruction set and a different machine code, and the processor wouldn’t understand the instructions.
- Let’s now look at the disassembly of the **.text** section, shown in three columns. The rightmost column is the actual assembly instructions, consisting of opcode and arguments. Notice that the instructions may appear differently here than they did in the original assembly source, since they are reconstructing from their binary representation alone. In particular, note that where in the source we referred to the symbols **hello_text** and **hello_text_len**, the symbols have been replaced by their numeric values: the memory address of the string and its actual length.

Furthermore, the concrete syntax used by `objdump` differs somewhat from the syntax we used for our `nasm`-compatible source code. For example, `objdump` usually will provide explicit typecasts for all memory accesses, such as `mov BYTE PTR [eax], dl`, indicating a byte-sized memory access; whereas we would probably write simply `mov [eax], dl`, and the size of the memory operation is implicit in the use of the byte-sized `dl` register.

- The middle column of the disassembly gives the hexadecimal encoding of each instruction. For example, in the first instruction, which puts the value 4 into the register `eax`, the encoding is five bytes: `b8 04 00 00 00`. The first byte identifies this as a `mov` instruction into `eax`, and the following four bytes are the 32-bit value 4 in little-endian order.

Note that unlike E15, different instructions are represented in hex by a different number of bits. Some instructions can be represented in only one byte, whereas some require up to fifteen. In this small program, we see examples of instructions in 2 bytes and 5 bytes.

- Finally, the leftmost column is the address at which the corresponding instruction begins. That is, the program expects that the OS will copy the instructions into memory at the given addresses.

In addition to the `.text` section, our program also has a `.data` section, which stores initialized data used by the program. We can request a dump of this section using the following command:

```
objdump -s -j .data helloworld
```

Your output should look like this:

```
helloworld:      file format elf32-i386
```

```
Contents of section .data:
```

```
80490a4 48656c6c 6f2c2077 6f726c64 0a      Hello, world.
```

This is a three-column output, with the ASCII data on the right, its hex values in the middle, and its address on the left. Notice that the address of the beginning of the string (0x80490a4) matches the address loaded into register `ecx` in the disassembly.

Out of interest, we can also get a disassembly of the data section with the following command:

```
objdump -D -Intel helloworld
```

Note that in addition to the `.text` section, the above command will attempt to disassemble the `.data` section as well. The result, while legal assembly language instructions, is nonsense: it's the result of attempting to interpret a sequence of bytes as code, when in reality it is not code. Still, I mention this because I wanted to demonstrate that the disassembly operation can apply to any sequence of bytes, whether or not it's really meaningful code.

2.5 Debugging

A *debugger* is a program that allows the analysis of another program, the *target program*, while the target is running. A debugger lets us walk through each instruction in the program, one at a time, while viewing the state of registers and variables. We'll be using a debugger named `gdb`.

To start our `helloworld` as a target under `gdb`, run the following command:

```
gdb helloworld
```

You will see the `>>>` prompt, indicating that we can now give commands to `gdb`.

Initially, the target program is loaded but not running. To start it, give the following command:

```
run0
```

(If you get an “Undefined command” message in response to `run0`, then you haven’t installed my `.gdbinit` correctly. If you get an “Entrypoint not found” message, then you probably haven’t correctly loaded your program.)

You will probably also want to enlarge your terminal window, to ensure that you can see all the stuff that `gdb` is trying to show you.

With `gdb` running our `helloworld` program, this is what I see:

```

— Output/messages —
Breakpoint 1, 0x08049000 in _start ()
— Assembly —
0x08049000 _start+0  mov     eax,0x4
0x08049005 _start+5  mov     ebx,0x1
0x0804900a _start+10 mov     ecx,0x804a000
0x0804900f _start+15 mov     edx,0xd
— Expressions —
— History —
— Memory —
— Registers —
eax 0x00000000    ecx 0x00000000    edx 0x00000000    ebx 0x00000000
esp 0xffffd2a0    ebp 0x00000000    esi 0x00000000    edi 0x00000000
eip 0x08049000    eflags [ IF ]      cs 0x00000023      ss 0x0000002b
ds 0x0000002b     es 0x0000002b      fs 0x00000000      gs 0x00000000
— Source —
— Stack —
[0] from 0x08049000 in _start+0
(no arguments)
— Threads —
[1] id 124229 name helloworld from 0x08049000 in _start+0
>>> █

```

Some things worth mentioning in the `gdb` display:

- In the “Output/messages” section, it will display anything that the target program prints.
- The “Assembly” section shows a disassembly of the program, with the next instruction highlighted.
- The “Registers” section shows the current value of all the registers.

Now, run `stepi` to execute the next instruction, which should be `mov eax, 0x4`. Notice that the displayed value of the `eax` register changes, and the subsequent instruction is highlighted. By running `stepi` repeatedly, we can trace the execution of the whole program.

When we reach the first `int 0x80` instruction, indicating the system call to display the string, it will be visible in the “Output/message” section.

Instead of stepping through the program instruction by instruction, we can set a *breakpoint*, that is, mark a particular instruction such that when it is executed, normal execution will stop and control will be returned to the debugger. You’ll need to know the address or label of the instruction you want to break on. In this program, for example, the last instruction (the `int` that invokes the `exit` syscall) is at address `0x080480a0`, so we can do `b *0x080480a0` to set a breakpoint there, and then `continue` to let the program run normally until it hits a breakpoint.

In `gdb`, you can use the `disassemble` command to show disassembled instructions and their addresses.

2.6 Syscall analysis

Another way to examine the workings of a program as it runs is to use **strace**. (Install it with the command `sudo apt-get install strace`.) The tool will output a line each time a given program invokes a syscall. I run it on our `helloworld` like this:

```
strace ./helloworld
```

That command generates this output:

```
execve("./helloworld", ["/helloworld"], [/* 49 vars */]) = 0
strace: [ Process PID=312 runs in 32 bit mode. ]
write(1, "Hello, world\n", 13)          = 13
exit(0)                                = ?
+++ exited with 0 +++
```

Notice that each line corresponds to a syscall: **execve** actually starts the program, the **write** call puts the text on the screen, and **exit** ends the program. **strace** uses a syntax similar to C to display each syscall.

2.7 Patching

It's possible to modify an executable binary program in order to change its behavior. **ghex** or **hexedit** is a good tool for this. Just changing string data is easy; for example, it would be trivial to change our `helloworld` program such that instead of "Hello, world" it prints out "Noodle, world." Changing a program's instructions is a little trickier: you'll need to know the encoding of the instruction that you want to change, and the encoding of the instruction you want to replace it with.

Often, the desired effect can be achieved by simply removing instructions, but this is not so straightforward: actually removing an instruction's bytes from the executable would mean changing the address of all subsequent instructions (as everything would be closer to the beginning of the file), which would break the program in various ways: all code referring to any address numerically higher than the point of removal would need to be adjusted. Instead, it's often sufficient to replace an undesirable instruction by one or several no-op instructions (**nop**, `0x90`). No-op does nothing, other than advance the program counter.

For example, let's say we want to prevent our `helloworld` program from actually displaying its message. All we have to do is remove the `int 80h` instruction that invokes the **write** syscall. From looking at the disassembly, we can see that this instruction is encoded as `cd 80` in hex, and that the bytes immediately after it are `b8 01`. If we open the `helloworld` binary in **ghex** or **hexedit**, we can search for the four bytes `cd 80 b8 01`, and change the first two to `90 90`. We've just overwritten that instruction with nops. Save our changes and run the program: no message is displayed.

2.8 Analyzing programs written in other languages

In this tutorial, we've described the process for reverse engineering a trivially simple program originally written in assembly language, but the tools we've used have a much broader application. You can apply the same tools (**objdump**, **gdb**, **strace**, etc) to any executable program, written in any language, on (most) any operating system. For example, let's write a simple program in C called `test.c`:

```
#include <stdio.h>
int main() {
    printf("Hello, world (in C)\n");
    return 0;
}
```

We can ask our C compiler to generate assembly language output to a file named `test.s` with the following command:

```
gcc -S -m32 test.c
```

Alternatively, we can compile the above program into an executable binary named `test`:

```
gcc -o test -m32 test.c
```

We can run the program:

```
./test
```

We can now do `objdump -d -Intel test` to get a disassembly of the executable binary. You may be surprised by the length and complexity of the output, compared to our straightforward assembly language version. If you load this program into `gdb`, you will notice that you can see the source code and access variables and functions by their name.

We can even disassemble and debug programs for which we don't have the source code. For example, the Linux command `ls` is used to display the contents of a directory. The executable file for this command is in `/bin/ls`. We can get a disassembly just as you would expect: `objdump -d -Intel /bin/ls`

By the way, we can use `strace` on programs such as `ls`. Try it; you may be surprised by the quantity of syscalls such an apparently simple program generates.

What about Python, or Ruby, or Perl? These languages don't have an explicit compilation step, as does C. Instead, you just "run" the source code. Are they translated to machine language, as C programs are? How can we analyze them?

These languages (and others) are called *interpreted* languages, because they require an *interpreter* to run. An interpreter is a program that can read the source code of the given language and carry out the instructions. (Interpreters may internally translate the code to machine language or some intermediary form, but typically that translated form is not written to a file and does not obviate the need for the interpreter.)

Let's save the following code into a Python program named `test.py`:

```
def main():
    print("Hello, world (in Python)")

main()
```

To run the program, we need to invoke the Python interpreter, telling it to run the code we've written. We type:

```
python test.py
```

Notice that we've run the Python interpreter, a program named `python`, which must be installed on your computer before you can run Python programs. The Python interpreter is in turn written in C, and compiled into a machine language binary. So you see, even if our program isn't translated to machine language, we still eventually need a machine language program in order to run it.

Our familiar tools will still work. For example, we can see the syscalls invoked by our program like this:

```
strace python test.py
```

On the other hand, since we need the Python source code in order to run a Python program, the need to reverse engineer Python programs at the machine language level is reduced.

2.9 Analysis on other operating systems

The tools we've discussed so far apply to Linux, but the underlying techniques work in any operating system, assuming you have the tools.

In MacOS, the equivalent of `objdump` is `otool`. You can use it to examine and disassemble any arbitrary executable on your computer. For example, to see how the built-in TextEdit program works, run this command:

```
otool -tV /Applications/TextEdit.app/Contents/MacOS/TextEdit
```

The disassembly from `otool` uses the so-called AT&T syntax, which, among other differences, reverses the order in which arguments are written after opcodes. A more advanced disassembler for MacOS is Hopper.

The MacOS equivalent of `strace` is called `dtruss`. You can watch the syscalls performed by the built-in `ls` command like this:

```
sudo dtruss ls
```

`gdb`, `nasm`, and other tools are also available on MacOS (although you may have to install them). Note that our `helloworld.asm` program won't work on MacOS, because the syscalls are different, even though the assembly language is the same.

There are similar tools available for Windows, although most come with Visual Studio, and aren't installed by default. If you've installed Visual Studio or the Windows SDK, you can try `DUMPBIN`, which does roughly what `objdump` does. `WinDbg` is the equivalent of `gdb`.

There are also third-party debuggers and binary analyzers. They provide much richer information and make it much easier to investigate programs, but they can be expensive. Examples include Binary Ninja, ImHex, ReHex, 010 Editor, Ghidra (developed by the NSA), and IDA Pro.

3 gdb commands

Some useful `gdb` commands:

- `run` – run the currently loaded program
- `continue` – continue running the currently loaded program until it either ends or hits a breakpoint
- `run0` – start running the currently loaded program, but stop at the first assembly instruction; this command requires our custom `.gdbinit`
- `stepi` – run the next assembly instruction; if the next instruction is a function call, move into that function and examine its instructions
- `nexti` – run the next assembly instruction; if the next instruction is a function call, execute the function but don't examine its instructions
- `b` – set a breakpoint at a particular location; if execution reaches the given address, execution will stop
 - `b 0x080480a0` – set a breakpoint at address 0x080480a0
- `i b` – list all breakpoints
- `d` – remove existing breakpoint
- `p` – print the value of an expression
 - `p $eax` – print the current value of the `eax` register
 - `p $eax+$ebx+2` – print the current value of the sum of the `eax` and `ebx` registers plus 2
 - `p *($eax+$ebx+2)` – print the value of the memory address pointed to by the sum of the `eax` and `ebx` registers plus 2
- `x` – examine the value at a memory location
 - `x/x 0x080480a0` – Show the data at the given address as a hex word.
 - `x/xb $ecx` – Show as a hex byte the data at the address stored in `ecx`.
 - `x/s 0x080480a0` – Show the data at the given address as a string.
- `quit` – leave `gdb`

Please also refer to the `gdb` Cheat Sheet.